

Universidad Internacional del Ecuador



INTRODUCCION A LAS REDES DE DATOS

SISTEMA DE GESTIÓN DE E-COMMERCE Desarrollado en Go con Programación

Funcional Aplicando Arrays, Slices y Maps - Unidad 1

Asignatura: Aprendizaje Autónomo 1 - Selección de Sistemas de Gestión Empresarial

Estudiante: Elizabeth Cardona

Carrera: Ingeniería en Ciberseguridad

Universidad: Universidad Internacional del Ecuador (UIDE)

Fecha de entrega: 24 de mayo de 2026

Modalidad: Trabajo Individual

Índice

1. INTRODUCCIÓN	3
2. OBJETIVO GENERAL Y ESPECÍFICOS	3
2.1 Objetivo General.....	3
2.2 Objetivos Específicos	3
3. MÓDULOS DEL SISTEMA Y ALCANCE	4
4. CONEXIÓN EXPLÍCITA CON UNIDAD 1: ARRAYS, SLICES, MAPS.....	4
4.1 Arrays en Go: Categorías Fijas.....	4
4.2 Slices en Go: Colecciones Dinámicas	5
4.3 Maps en Go: Búsquedas Eficientes	5
5. ESTRUCTURA FUNCIONAL DEL SOFTWARE EN GO	6
5.1 Arquitectura Modular	6
5.2 Flujo Funcional de un Pedido	7
6. ENFOQUE DE PROGRAMACIÓN FUNCIONAL.....	7
7. RELACIÓN INTERDISCIPLINARIA	8
8. PAQUETES Y LIBRERÍAS UTILIZADAS	9
9. ALCANCE Y LIMITACIONES DEL PROYECTO	9
9.1 Alcance	9
9.2 Limitaciones	10
9.3 Propuestas de Mejora Futura	10
10. CONCLUSIONES	11
REFERENCIAS	12

1. INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un Sistema de Gestión de E-commerce utilizando el lenguaje de programación Go y aplicando explícitamente los conceptos de la Unidad 1: Arrays, Slices y Maps, combinados con principios de Programación Funcional. Este sistema busca automatizar los procesos clave de una tienda en línea enfocada en productos ecuatorianos como artesanías, café de altura y chocolate artesanal. Los beneficios principales son: mejorar la eficiencia operativa, reducir errores mediante el uso de inmutabilidad y facilitar la toma de decisiones a través de reportes funcionales.

El trabajo aplica los conocimientos adquiridos en la Unidad 1 sobre Estructuras de Datos en Go, combinándolos con técnicas modernas de desarrollo de software funcional para crear una solución mantenible, testeable y escalable.

2. OBJETIVO GENERAL Y ESPECÍFICOS

2.1 Objetivo General

Desarrollar un sistema de gestión de e-commerce en Go que permita administrar productos, usuarios, carrito de compras y pedidos mediante programación funcional, aprovechando las estructuras de datos de la Unidad 1 (Arrays, Slices, Maps) para garantizar código limpio, mantenible, predecible y eficiente en memoria.

2.2 Objetivos Específicos

- Implementar los módulos principales de un sistema de gestión empresarial utilizando estructuras de Go: Arrays para categorías fijas, Slices para colecciones dinámicas y Maps para búsquedas eficientes.
- Aplicar conceptos de programación funcional: funciones puras, inmutabilidad, higher-order functions (FilterBy, MapTransform, Reduce) y composición.
- Documentar la estructura y funcionalidades del sistema, explicitando la conexión pedagógica con los temas de la Unidad 1.
- Demostrar la integración interdisciplinaria entre gestión empresarial, ciencias de la computación y matemáticas aplicadas.
- Validar el funcionamiento mediante ejecución en consola y demostración en video.

3. MÓDULOS DEL SISTEMA Y ALCANCE

Modulo	Responsabilidad Funcional	Principio FP Aplicado	Alcance
Gestión de Productos	Administración completa del catálogo de productos ecuatorianos	CRUD de productos, búsqueda por categoría, actualización de stock en tiempo real	Completo
Gestión de Usuarios	Control de acceso, roles y perfiles de clientes y administradores	Registro de usuarios, login simulado, diferenciación de roles (admin/cliente)	Completo
Carrito de Compras	Manejo temporal de selecciones del cliente antes de confirmar compra	Agregar items, eliminar items, actualizar cantidades, cálculo automático de total	Completo
Gestión de Pedidos	Procesamiento, seguimiento y historial de compras realizadas	Crear pedido desde carrito, cambiar estado (pendiente/enviado/entregado), historial de cliente	Completo
Reportes	Generación de información estratégica para toma de decisiones	Ventas totales por período, productos más vendidos, valor actual del inventario	Básico

4. CONEXIÓN EXPLÍCITA CON UNIDAD 1: ARRAYS, SLICES, MAPS

4.1 Arrays en Go: Categorías Fijas

Definición teórica (Unidad 1): Los arrays son colecciones de elementos del mismo tipo almacenados en memoria contigua, con tamaño fijo definido en tiempo de compilación.

Aplicación en el proyecto:

```
1 categorias := [3]string{"Artesanías", "Café", "Chocolate"}
```

Justificación pedagógica: Se utiliza un array de tamaño fijo para las categorías porque son un conjunto cerrado y conocido de antemano. Esto garantiza seguridad de tipos: el compilador de Go asegura que solo existan estas tres categorías, previniendo errores de escritura o valores inválidos en la gestión empresarial.

4.2 Slices en Go: Colecciones Dinámicas

Definición teórica (Unidad 1): Los slices son vistas dinámicas sobre arrays, que permiten crecer o reducirse sin copiar toda la estructura. Son referencias que apuntan a un array subyacente.

Aplicación en el proyecto:

```
1 type ShoppingCart []CartItem
2 carrito := make([]CartItem, 0)
3 carrito = append(carrito, nuevoItem)
```

Justificación pedagógica: El carrito de compras necesita agregar y eliminar productos dinámicamente según la interacción del usuario. Los slices permiten esta flexibilidad con eficiencia en memoria, ya que no copian el array completo al modificarlo. Además, funciones como `len()` y `cap()` permiten controlar el crecimiento, alineándose con buenas prácticas de gestión de recursos.

4.3 Maps en Go: Búsquedas Eficientes

Definición teórica (Unidad 1): Los maps son estructuras de datos que asocian claves únicas con valores, implementadas como tablas hash para búsquedas en tiempo constante $O(1)$.

Aplicación en el proyecto:

```
1 type ProductCatalog map[string]Product
2 producto := catalogo["CAF-001"]
```

Justificación pedagógica: En un e-commerce real, los usuarios buscan productos por ID, nombre o categoría. Los maps permiten recuperar un producto en tiempo constante sin recorrer listas completas, lo que es crítico para la escalabilidad. La clave string (ID del producto) garantiza unicidad, mientras que el valor `Product` contiene toda la información empresarial relevante.

Estructura Unidad 1	Sintaxis en Go	Caso de Uso en el Proyecto	Ventaja Empresarial
Array	[n]Tipo	Categorías fijas [3]string	Seguridad de tipos, sin valores inválidos
Slice	[]Tipo	Carrito de compras []CartItem	Flexibilidad para agregar/eliminar dinámicamente
Map	map[Clave]Valor	Catálogo map[string]Product	Búsqueda instantánea, escalabilidad

5. ESTRUCTURA FUNCIONAL DEL SOFTWARE EN GO

5.1 Arquitectura Modular

El sistema sigue una arquitectura modular que separa responsabilidades y facilita el mantenimiento:

- main.go: Punto de entrada de la aplicación. Inicializa el catálogo de productos, crea el carrito vacío y ejecuta la demostración de funcionalidades.
- modules/products.go: Contiene la lógica de gestión de productos. Utiliza map[string]Product para el catálogo y funciones puras para CRUD.
- modules/cart.go: Implementa el carrito de compras como slice dinámico. Incluye funciones puras para agregar items y calcular totales.
- modules/reports.go: Genera reportes utilizando funciones de orden superior como Reduce para agregaciones.
- utils/functional.go: Biblioteca de funciones puras reutilizables: FilterBy, MapTransform y Reduce.

- `data/products.json`: Archivo de persistencia con datos de ejemplo de productos ecuatorianos.

5.2 Flujo Funcional de un Pedido

1. Cliente navega el catálogo (Map: búsqueda $O(1)$ por categoría).
2. Selecciona productos y los agrega al carrito (Slice: append sin mutar original).
3. El sistema calcula el total (Reduce: función pura de acumulación)
4. Cliente confirma la compra
5. Se actualiza el inventario (Nuevo Map, sin modificar el original)
6. Se genera reporte de venta (Composición de funciones puras)

Este flujo garantiza que cada paso sea predecible, testeable y libre de efectos secundarios no deseados.

6. ENFOQUE DE PROGRAMACIÓN FUNCIONAL

Se aplicaron los siguientes principios de programación funcional:

- **Funciones Puras:** Todas las funciones de lógica de negocio (`CalculateTotal`, `FindByCategory`, `AddToCart`) devuelven el mismo resultado para los mismos parámetros y no producen efectos secundarios. Esto facilita pruebas unitarias y depuración.
- **Inmutabilidad:** Ninguna función modifica directamente sus parámetros de entrada. En su lugar, retornan nuevas estructuras con los cambios aplicados. Por ejemplo, `AddToCart` retorna un nuevo slice en lugar de modificar el carrito original.
- **Higher-Order Functions:** Se implementaron funciones que reciben otras funciones como parámetros: `FilterBy` (filtra con predicado), `MapTransform` (transforma elementos) y `Reduce` (acumula valores). Esto permite composición y reutilización.

- Composición de Funciones: La funcionalidad compleja se construye combinando funciones simples. Ejemplo: `GetItemsByCategory` compone `MapTransform` + `FilterBy` para filtrar items del carrito por categoría de producto.
- Ausencia de Efectos Secundarios en Lógica Principal: Las operaciones de I/O (lectura/escritura de JSON) están aisladas en la capa utils, mientras que la lógica de negocio permanece pura y predecible.

7. RELACIÓN INTERDISCIPLINARIA

Este proyecto integra conocimientos de múltiples disciplinas para crear una solución completa:

- Gestión Empresarial: Se modelan procesos reales de e-commerce como gestión de inventario, procesamiento de pedidos, roles de usuario y generación de reportes para toma de decisiones. Los conceptos de procesos transversales y flujos de negocio de la Unidad 1 se aplican directamente.
- Ciencias de la Computación: Se implementa el paradigma de programación funcional en Go, aprovechando estructuras de datos eficientes (arrays, slices, maps) y principios como inmutabilidad y composición para crear software robusto.
- Matemáticas Aplicadas: Las funciones de orden superior como `Reduce` implementan operaciones matemáticas de acumulación y agregación. El cálculo del total del carrito es una suma iterativa; el valor del inventario es una reducción con multiplicación.
- Diseño de Sistemas: La arquitectura modular sigue principios de separación de responsabilidades y cohesión, facilitando escalabilidad y mantenimiento futuro.

Esta combinación interdisciplinaria permite generalizar soluciones de gestión a diferentes contextos empresariales, demostrando que los fundamentos teóricos pueden adaptarse a problemas prácticos complejos.

8. PAQUETES Y LIBRERÍAS UTILIZADAS

Paquete / Librería	Uso en el Proyecto	Tipo
fmt	Formateo de salida en consola para demostración	Estándar de Go
encoding/json	Serialización y deserialización de productos a/from JSON	Estándar de Go
os	Lectura y escritura de archivos de datos	Estándar de Go
sort	Ordenamiento de reportes (productos más vendidos)	Estándar de Go

Nota: No se utilizaron paquetes de terceros para mantener la transparencia del aprendizaje, garantizar compatibilidad y enfocarse en los fundamentos de Go y programación funcional.

9. ALCANCE Y LIMITACIONES DEL PROYECTO

9.1 Alcance

- Backend funcional completo desarrollado en Go, ejecutable en consola.
- Implementación de los cinco módulos principales: productos, usuarios, carrito, pedidos y reportes.
- Persistencia de datos mediante archivos JSON que simulan una base de datos.
- Demostración explícita de conexión con los temas de la Unidad 1: Arrays, Slices y Maps.
- Aplicación de principios de programación funcional: funciones puras,

inmutabilidad y higher-order functions

- Documentación completa en README.md y este documento PDF.

9.2 Limitaciones

- Versión de consola: No incluye interfaz gráfica web ni aplicación móvil.
- Autenticación simulada: El login de usuarios no implementa cifrado real ni tokens JWT.
- Persistencia básica: Los datos se guardan en archivos JSON en lugar de una base de datos relacional.
- Sin pasarela de pagos: El proceso de compra se simula sin integración con servicios de pago reales.

9.3 Propuestas de Mejora Futura

- Implementar API REST con Gin o Echo para exponer funcionalidades vía HTTP.
- Desarrollar frontend con React o Vue.js para experiencia de usuario completa.
- Integrar base de datos PostgreSQL para persistencia robusta y consultas complejas.
- Agregar autenticación JWT y cifrado de contraseñas con bcrypt.
- Desplegar en la nube (AWS, GCP o Azure) con CI/CD automatizado.

10. CONCLUSIONES

El desarrollo de este Sistema de Gestión de E-commerce permitió aplicar de manera práctica y explícita los conceptos de la Unidad 1: Arrays, Slices y Maps del lenguaje Go. Cada estructura de datos fue seleccionada conscientemente según sus características técnicas y su idoneidad para resolver problemas específicos de gestión empresarial.

La combinación con programación funcional demostró ser una estrategia efectiva para crear código predecible, fácil de probar y mantener. La inmutabilidad previene errores sutiles de estado compartido, mientras que las funciones puras facilitan la composición y reutilización.

Este proyecto evidencia la capacidad de transferir conocimientos teóricos adquiridos en el aula a soluciones concretas del mundo real. La integración interdisciplinaria entre gestión empresarial, ciencias de la computación y matemáticas aplicadas enriquece el aprendizaje y prepara para desafíos profesionales complejos.

Finalmente, el enfoque modular y documentado del proyecto sienta las bases para futuras ampliaciones, demostrando que una buena arquitectura inicial permite escalar soluciones sin comprometer la calidad del código.

REFERENCIAS

- Macias, J. (2022). Estructuras de datos en Go: Arrays, Slices y Maps. Material de clase - Unidad 1, Universidad Internacional del Ecuador.
- McGrath, M. (2021). Go Programming in Easy Steps. In Easy Steps Limited. Capítulo 6: Build Structures, Capítulo 7: Create Arrays, pp. 116-157.
- D'Anna, J., et al. (2019). Programming in Go: Creating Applications for the 21st Century. Addison-Wesley Professional.
- Customero, R. (2010). Curso de Go. Universidad Politécnica de Madrid.
- Tipos de datos compuestos, pp. 52-58. Disponible en:
 - https://www.academia.edu/5180286/Curso_de_Go
- Documentación oficial de Go. (2026). Effective Go. https://go.dev/doc/effective_go
- Documentación oficial de Go. (2026). Go by Example: Maps, Slices. <https://gobyexample.com/>